

# Dokumentasi Proyek Chatbot Bantuan Pengguna AI

Dokumen ini memberikan gambaran teknis yang komprehensif mengenai proyek chatbot bantuan pelanggan. Proyek ini dibangun menggunakan Flask untuk backend, Qdrant sebagai database vektor untuk fungsionalitas RAG (Retrieval-Augmented Generation), dan MySQL untuk menyimpan data terstruktur seperti laporan *error* dan isu pengguna.

## 1. Arsitektur Proyek

Arsitektur sistem ini terdiri dari beberapa komponen utama yang saling berinteraksi:

1. Aplikasi Flask (app.py): Bertindak sebagai *backend* utama yang menyediakan API *endpoint* untuk antarmuka (*frontend*) dan mengelola logika bisnis.
2. Database Relasional (MySQL): Digunakan untuk menyimpan data terstruktur, seperti laporan *error* dari sistem dan isu yang dilaporkan oleh pengguna.
3. Database Vektor (Qdrant): Menyimpan *vector embeddings* dari basis pengetahuan (*knowledge base*) dan video tutorial. Ini memungkinkan pencarian semantik yang efisien untuk menjawab pertanyaan pengguna.
4. Logika Inti (LangchainSpil.py & EditKnowledge.py):
  - LangchainSpil.py berisi logika *agent* yang menggunakan LLM (Large Language Model) dan *tools* yang tersedia untuk memproses permintaan pengguna.
  - EditKnowledge.py menyediakan fungsi-fungsi untuk mengelola (CRUD - *Create, Read, Update, Delete*) data di Qdrant dan MySQL.
5. Orkestrasi Kontainer (Docker): docker-compose.yml dan Dockerfile digunakan untuk mengelola, membangun, dan menjalankan semua layanan (Flask, MySQL, Qdrant) secara konsisten di lingkungan yang terisolasi.

## 2. Struktur File Proyek

Berikut adalah penjelasan untuk setiap file dalam proyek:

| Nama File        | Deskripsi                                                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app.py           | File utama aplikasi Flask. Berisi semua <i>API endpoint</i> untuk <i>chat</i> , manajemen basis pengetahuan, video tutorial, serta laporan <i>error</i> dan isu pengguna. |
| LangchainSpil.py | Mengandung logika inti dari <i>agent</i>                                                                                                                                  |

|                    |                                                                                                                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    | LangChain, termasuk definisi <i>tools</i> (RAG, <i>web scraping</i> , interaksi database), <i>prompting</i> , dan <i>chain</i> pemrosesan yang menentukan alur kerja <i>chatbot</i> . |
| EditKnowledge.py   | Berisi fungsi-fungsi pembantu untuk berinteraksi langsung dengan Qdrant (menambah, mengubah, menghapus data pengetahuan) dan MySQL (mengelola data <i>error</i> dan isu).             |
| inisiat.py         | Skrip yang dijalankan sekali untuk menginisialisasi koleksi di Qdrant dan mengisi data awal ( <i>dummy data</i> ) ke dalam Qdrant dan MySQL.                                          |
| docker-compose.yml | Mendefinisikan dan mengonfigurasi layanan yang dibutuhkan proyek: app (aplikasi Flask), db (MySQL), dan qdrant.                                                                       |
| Dockerfile         | Berisi instruksi untuk membangun <i>image</i> Docker untuk aplikasi Flask, termasuk instalasi dependensi dari requirements.txt.                                                       |
| .env               | File konfigurasi untuk menyimpan variabel lingkungan yang sensitif seperti kredensial database dan kunci API.                                                                         |
| requirements.txt   | Mendaftar semua pustaka Python yang dibutuhkan oleh proyek.                                                                                                                           |
| sql table.sql      | Skrip SQL untuk membuat database Report dan tabel error serta User_issue di MySQL.                                                                                                    |

### 3. Konfigurasi Lingkungan (.env)

File ini menyimpan konfigurasi yang digunakan oleh layanan lain.

```
DB_USER="root"
DB_PASSWORD="rootroot"
DB_HOST="localhost"
DB_NAME="Report"
QDRANT_HOST="qdrant"
OPENAI_API_KEY="DUMMY"
```

- DB\_\*: Kredensial untuk koneksi ke database MySQL.
- QDRANT\_HOST: Nama *host* dari layanan Qdrant, yang akan diresolusi oleh Docker.
- OPENAI\_API\_KEY: Kunci API untuk model bahasa. Dalam kasus ini, diisi "DUMMY" karena menggunakan *endpoint* kustom.

### 4. Dependensi (requirements.txt)

Proyek ini bergantung pada beberapa pustaka Python utama:

- Flask: Kerangka kerja web untuk membangun API.
- qdrant-client: Klien untuk berinteraksi dengan database vektor Qdrant.
- sentence-transformers: Untuk membuat *vector embeddings* dari teks.
- langchain: Kerangka kerja utama untuk membangun aplikasi berbasis LLM.
- mysql-connector-python: Driver untuk menghubungkan aplikasi dengan database MySQL.
- requests, beautifulsoup4: Untuk melakukan *web scraping*.
- gunicorn: WSGI server untuk menjalankan aplikasi Flask di lingkungan produksi.

### 5. Setup dan Instalasi

Proyek ini dirancang untuk dijalankan menggunakan Docker dan Docker Compose, yang menyederhanakan proses *setup*.

Langkah-langkah:

1. Pastikan Docker Terinstal: Docker dan Docker Compose harus sudah terinstal di sistem Anda.
2. Buat File .env: Salin konten dari contoh di atas dan sesuaikan jika perlu.
3. Jalankan Docker Compose: Buka terminal di direktori utama proyek dan jalankan

perintah berikut:

```
docker-compose up --build
```

4. Inisialisasi Data (Opsional): Jalankan skrip inisiate.py untuk mengisi data awal.  
`docker-compose exec flask_app python inisiate.py`

5. Akses Aplikasi:

- API Chatbot: `http://localhost:5000`
- Panel Kontrol: `http://localhost:5000/control`
- Qdrant UI: `http://localhost:6334/dashboard`

## 6. Skema Database (sql table.sql)

Database Report digunakan untuk mencatat masalah yang tidak dapat ditangani oleh *chatbot*.

- Tabel error: Menyimpan *error* internal yang terjadi pada sistem.
- Tabel User\_issue: Menyimpan isu atau pertanyaan dari pengguna yang tidak dapat dijawab oleh *chatbot*.

## 7. Penjelasan Fungsi Secara Rinci

Bagian ini menguraikan fungsionalitas dari setiap fungsi penting dalam kode sumber proyek.

app.py - Backend & API Endpoint

File ini bertanggung jawab untuk menyediakan semua *endpoint* yang diperlukan oleh *frontend*.

| Endpoint | Metode | Fungsi  | Deskripsi                                                                   |
|----------|--------|---------|-----------------------------------------------------------------------------|
| /        | GET    | index() | Menyajikan halaman HTML utama untuk antarmuka <i>chat</i> .                 |
| /chat    | POST   | chat()  | Menerima pesan (teks dan/atau gambar) dari pengguna, memanggil <i>chain</i> |

|                      |        |                           |                                                                                      |
|----------------------|--------|---------------------------|--------------------------------------------------------------------------------------|
|                      |        |                           | LangChain untuk mendapatkan respons, dan mengembalikannya sebagai JSON.              |
| /control             | GET    | knowledge_control_index() | Menyajikan halaman HTML untuk panel kontrol manajemen pengetahuan.                   |
| /knowledge           | GET    | get_all_knowledge()       | Mengambil semua data pengetahuan dari Qdrant dan menyajikannya dalam format tabel.   |
| /knowledge           | POST   | add_knowledge_route()     | Menambahkan data pengetahuan baru ke Qdrant berdasarkan input dari <i>frontend</i> . |
| /knowledge/<id_docs> | PUT    | edit_knowledge_route()    | Memperbarui data pengetahuan yang ada di Qdrant.                                     |
| /knowledge/<id_docs> | DELETE | delete_knowledge_route()  | Menghapus satu set data pengetahuan dari Qdrant berdasarkan ID                       |

|                    |        |                      |                                                                              |
|--------------------|--------|----------------------|------------------------------------------------------------------------------|
|                    |        |                      | dokumennya.                                                                  |
| /videos            | GET    | get_all_videos()     | Mengambil semua data video tutorial dari Qdrant.                             |
| /videos            | POST   | add_video_route()    | Menambahkan data video tutorial baru ke Qdrant.                              |
| /videos/<point_id> | PUT    | edit_video_route()   | Memperbarui data video tutorial yang ada.                                    |
| /videos/<point_id> | DELETE | delete_video_route() | Menghapus data video tutorial berdasarkan ID uniknya.                        |
| /error             | GET    | get_all_errors()     | Mengambil semua laporan <i>error</i> dari database MySQL.                    |
| /error             | POST   | add_error_route()    | Menambahkan laporan <i>error</i> baru (biasanya dilaporkan oleh sistem/LLM). |
| /error/<error_id>  | PUT    | edit_error_route()   | Mengubah status atau detail dari laporan <i>error</i> yang ada.              |
| /error/<error_id>  | DELETE | delete_error_route() | Menghapus laporan <i>error</i> dari database.                                |

|                   |        |                      |                                                                                         |
|-------------------|--------|----------------------|-----------------------------------------------------------------------------------------|
| /issue            | GET    | get_all_issues()     | Mengambil semua laporan isu pengguna dari MySQL.                                        |
| /issue            | POST   | add_issue_route()    | Menambahkan laporan isu baru (biasanya dilaporkan oleh LLM ketika tidak bisa membantu). |
| /issue/<issue_id> | PUT    | edit_issue_route()   | Mengubah status atau detail dari laporan isu yang ada.                                  |
| /issue/<issue_id> | DELETE | delete_issue_route() | Menghapus laporan isu dari database.                                                    |

### EditKnowledge.py - Manajemen Data

Berfungsi sebagai lapisan abstraksi untuk berinteraksi dengan database Qdrant dan MySQL.

#### Fungsi Manajemen Pengetahuan (Qdrant)

- **get\_all\_points\_and\_group\_by\_id\_docs():** Mengambil semua data poin dari koleksi Qdrant, lalu mengelompokkannya berdasarkan id\_docs untuk menyatukan kembali *chunks* yang terpisah. Fungsi ini untuk menampilkan knowledge chatbot.
- **convert\_to\_tabular():** Mengubah data yang sudah dikelompokkan menjadi format daftar (list) yang sederhana agar mudah ditampilkan di *frontend*.
- **chunking\_txt():** Memecah sebuah teks panjang menjadi beberapa bagian (*chunks*) yang lebih kecil menggunakan RecursiveCharacterTextSplitter. Ini adalah langkah penting sebelum membuat *embeddings*.
- **add\_to\_qdrant\_Chunck\_Knowledge():** Menerima *chunks* teks, membuat *vector embeddings* untuk setiap *chunk*, dan menyimpannya ke Qdrant sebagai PointStruct.

- **delete\_by\_id\_docs\_knowledge():** Menghapus semua *chunks* yang memiliki *id\_docs* yang sama.
- **edit\_knowledge():** Mengimplementasikan logika edit dengan cara menghapus data lama terlebih dahulu, lalu menambahkan data yang baru.
- **add\_knowledge():** Menyederhanakan proses penambahan pengetahuan baru dengan memanggil fungsi *chunking\_txt* dan *add\_to\_qdrant\_Chunck\_Knowledge*.

#### Fungsi Manajemen Video (Qdrant)

- **get\_all\_data\_as\_dataframe():** Mengambil semua data dari koleksi video tutorial dan mengubahnya menjadi DataFrame Pandas untuk kemudahan manipulasi.
- **add\_to\_qdrant\_tutorialVid():** Menambahkan data video tutorial baru ke Qdrant.
- **delete\_point\_by\_id():** Menghapus satu data video berdasarkan ID uniknya.
- **edit\_vid():** Mengimplementasikan logika edit dengan menghapus data lama dan menambahkan yang baru.

#### Fungsi Manajemen Error & Isu (MySQL)

- **create\_db\_connection():** Fungsi utilitas untuk membuat dan mengembalikan koneksi ke database MySQL menggunakan konfigurasi dari *.env*.
- **get\_all\_errors() / get\_all\_issues():** Mengambil semua baris data dari tabel *error* atau *user\_issue*.
- **add\_error() / add\_issue():** Menambahkan baris data baru ke tabel yang sesuai.
- **edit\_error() / edit\_issue():** Memperbarui baris data yang ada berdasarkan ID-nya.
- **delete\_error\_by\_id() / delete\_issue\_by\_id():** Menghapus baris data dari tabel berdasarkan ID-nya.

#### LangchainSpil.py - Otak Chatbot

File ini berisi logika inti dari agen LLM.

- **SimpleEndpointChatModel:** Kelas kustom yang diwariskan dari *ChatOpenAI*. Tujuannya adalah untuk meng-override metode *\_generate* agar dapat mengirim permintaan ke *endpoint* LLM lokal, bukan ke API OpenAI resmi.
- **Tools:**
  - **qdrant\_rag\_tool():** Mencari informasi tekstual yang relevan dari basis pengetahuan di Qdrant berdasarkan *query* pengguna.
  - **qdrant\_rag\_tool\_vid():** Mencari link video tutorial yang relevan dari Qdrant.
  - **fetch\_navigation\_structure() / retrieve\_website\_content():** Fungsi untuk melakukan *web scraping*. Satu untuk struktur navigasi, satu lagi untuk konten teks.
  - **add\_eror\_to\_db() / add\_user\_isue\_to\_db():** Fungsi yang diekspos sebagai

*tool* agar LLM dapat secara mandiri melaporkan *error* atau isu pengguna ke database MySQL.

- Agent: Dibuat menggunakan `initialize_agent` dengan tipe `zero-shot-react-description`. Ini memungkinkan LLM untuk "berpikir" (*Thought*), memilih tindakan (*Action*), dan mengeksekusi *tool* (*Action Input*) untuk menyelesaikan permintaan pengguna.
- **`analyze_base64_image()`**: Fungsi yang dipanggil ketika input berisi gambar. Fungsi ini mengirimkan gambar (dalam format base64) ke LLM untuk dianalisis.
- `router_chain`: Sebuah `RunnableBranch` yang berfungsi sebagai "router". Ia memeriksa apakah input dari pengguna menyertakan gambar.
  - Jika ya, ia akan menjalankan alur `image_processing_flow` (analisis gambar lalu panggil agen).
  - Jika tidak, ia akan menjalankan alur `text_only_flow` (langsung panggil agen dengan teks).

`inisiat.py` – Skrip Inisialisasi

Skrip ini dijalankan untuk mengisi database dengan data awal.

- **`read_and_chunk_txt()`**: Membaca konten dari file `.txt` lokal dan langsung memecahnya menjadi *chunks*.
- **`add_to_qdrant_Chunck()` / `add_to_qdrant_tutorialVid()`**: Fungsi pembantu di dalam skrip ini untuk menambahkan data pengetahuan dan video ke Qdrant.
- Alur Utama: Skrip ini pertama-tama membuat koleksi di Qdrant, lalu membaca file-file pengetahuan dari direktori RAG Knowledge, dan terakhir menambahkan data *dummy* untuk *error* dan isu ke MySQL.